

GAUSS SEIDEL

MÉTODOS NUMÉRICOS

Contenido

- 01 Objetivo
- 02 Descripción
- 03 Pseudocódigo
- 04 Animación
- 05 Código
- 06 Casos de Prueba
- 07 Conclusión

Objetivo



Comprender paso a paso el método de Gauss-Seidel mediante una explicación visual y estructurada hasta su implementación computacional que facilite el aprendizaje del proceso iterativo y su implementación práctica.

Descripción

El método consiste en hacer iteraciones a partir de un vector inicial, para encontrar los valores de las incógnitas hasta llegar a una tolerancia deseada, la diferencia radica en que cada vez que se desee encontrar un nuevo valor de una x_i , además de usar los valores anteriores de las x , también utiliza valores actuales de las x encontradas antes (desde x_0 hasta x_{i-1}). La ecuación es la siguiente:

$$x_i^{(k)} = \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^k - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right) / a_{ii}$$

Ventajas

1. Mayor rapidez de convergencia que Jacobi
 - Usa inmediatamente los valores recién calculados dentro de la misma iteración, lo que generalmente acelera la convergencia.
2. Simplicidad de implementación
 - Es un método iterativo directo y fácil de programar, especialmente para sistemas grandes.

Ventajas

1. Menor uso de memoria
 - No requiere almacenar múltiples matrices adicionales; solo se actualiza el vector solución.
2. Eficiente para matrices grandes y dispersas
 - Resulta práctico en problemas de ingeniería donde los sistemas son extensos.

Desventajas

- “La desventaja del método de Gauss-Seidel es que no siempre converge, o algunas veces lo hace de manera lenta sobre la solución verdadera. Es muy confiable sólo para aquellos sistemas que son diagonalmente dominantes.”(Chapra & Canale, 2015)
- “Cuando la matriz en cuestión es muy grande y esparcida (es decir, cuando la mayoría de los elementos son cero), los métodos de eliminación desperdician grandes cantidades de memoria de cómputo al guardar ceros.”(Chapra & Canale, 2005).

Desventajas

- “El número de operaciones necesarias para resolver un sistema crece rápidamente conforme aumenta el tamaño del sistema” (Chapra & Canale, 2015).
- En algunos casos, es necesario aplicar técnicas de pivoteo para evitar errores numéricos significativos durante el proceso de eliminación (Chapra & Canale, 2015).

Algoritmo

- 1: Plantear el sistema de ecuaciones lineales en forma matricial y establecer las condiciones iniciales del método
- 2: Normalizar cada fila dividiendo sus valores por el elemento de la diagonal principal.
- 3: Simplificar las ecuaciones para que el coeficiente de cada incógnita sea 1.
- 4: Realizar un primer cálculo rápido de las variables para tener un punto de partida.

Algoritmo

- 5: Generar valores base que servirán como "borrador" para las mejoras sucesivas.
- 6: Iniciar un ciclo repetitivo para recalcular cada variable y ganar precisión y aplicar λ para promediar el valor nuevo con el anterior.
- 7: Estabilizar la búsqueda del resultado evitando cambios bruscos en los números y validar la precisión.

Pseudocódigo

Fase 1: Normalización

El algoritmo recorre cada fila i y busca el valor en la diagonal principal ($a_{i,i}$)

ALGORITMO Gseid ($a, b, n, es, imax, lambda, x$)

// FASE 1: NORMALIZACIÓN

```
DOFOR i = 1, n
    dummy = a[i, i]
    DOFOR j = 1, n
        a[i, j] = a[i, j] / dummy
    END DO
    b[i] = b[i] / dummy
END DO
```

Pseudocódigo

Fase 2: Primera Aproximación

Se realiza un cálculo rápido para obtener un primer valor de x para cada incógnita. Se toma el valor de b_i y se le resta la suma de los otros elementos de sus respectivas x

```
DOFOR i = 1, n
  sum = b[i]
  DOFOR j = 1, n
    IF i ≠ j THEN
      sum = sum - a[i, j] * x[j]
    END IF
  END DO
  x[i] = sum
END DO
```

Pseudocódigo

Fase 3: El Ciclo de Iteración y la Relajación

Trabaja repetidamente para acercarse a la respuesta correcta:

- Primero calcula qué valor debería tener x_i
- Se aplica una formula

$$x_i = \lambda \cdot \text{sum} + (1 - \lambda) \cdot x_{i,\text{old}}$$

La cual hay tres factores.

```
iter = 1
DOFOR
  centinela = 1
  DOFOR i = 1, n
    old = x[i]
    sum = b[i]

    DOFOR j = 1, n
      IF i ≠ j THEN
        sum = sum - a[i, j] * x[j]
      END IF
    END DO

    // Aplicación de la técnica de relajación
    x[i] = lambda * sum + (1. - lambda) * old

    // Verificación de convergencia
    IF centinela = 1 AND x[i] ≠ 0 THEN
      ea = ABS((x[i] - old) / x[i]) * 100.
      IF ea > es THEN
        centinela = 0
      END IF
    END IF
  END DO
```

Pseudocódigo

Fase 3: El Ciclo de Iteración y la Relajación

- Si $\lambda = 1$, es un Gauss-Seidel normal.
- Si $\lambda > 1$, el algoritmo es "agresivo" y trata de llegar más rápido al resultado
- Si $\lambda < 1$, el algoritmo es "precavido" y avanza lento para evitar pasarse (Sub-relajación).

```
iter = 1
DOFOR
  centinela = 1
  DOFOR i = 1, n
    old = x[i]
    sum = b[i]

    DOFOR j = 1, n
      IF i ≠ j THEN
        sum = sum - a[i, j] * x[j]
      END IF
    END DO

    // Aplicación de la técnica de relajación
    x[i] = lambda * sum + (1. - lambda) * old

    // Verificación de convergencia
    IF centinela = 1 AND x[i] ≠ 0 THEN
      ea = ABS((x[i] - old) / x[i]) * 100.
      IF ea > es THEN
        centinela = 0
      END IF
    END IF
  END DO
```

Pseudocódigo

Fase 4: Control de Calidad y Parada

Como se asume que se esta realizando otro proceso pasa que:

- Se compara el valor nuevo de x_i con el valor anterior.
- Si el error es más grande ≥ 0 que la tolerancia permitida se vuelve a realizar el proceso.
- Si el error es ≤ 1 significa que todos los resultados son precisos y termina.

```
iter = iter + 1
```

```
// Condición de parada  
(convergencia o límite de iteraciones)
```

```
IF centinela = 1 OR (iter  $\geq$  imax) EXIT
```

```
END DO
```

Código en Python

```
# PASO 1: Definir la función y cargar los parámetros  
def Gseid(a, b, n, x, imax, es, lmbda):
```

```
# PASO 2: Normalizar cada fila dividiendo sus elemento  
    for i in range(n):  
        dummy = a[i][i]  
        for j in range(n):  
            a[i][j] = a[i][j] / dummy  
        b[i] = b[i] / dummy
```

```
# PASO 3: Simplificar las ecuaciones
```

Código en Python

```
# PASO 4: Realizar un primer cálculo básico para punto de
partida
for i in range(n):
    suma = b[i]
    for j in range(n):
        if i != j:
            suma = suma - a[i][j] * x[j]
    x[i] = suma
```

Código en Python

PASO 5: Iniciar las Iteraciones

```
iter = 1
```

```
while True:
```

```
    centinela = 1
```

```
    for i in range(n):
```

```
        old = x[i]
```

```
        suma = b[i]
```

```
        for j in range(n):
```

```
            if i != j:
```

```
                suma = suma - a[i][j] * x[j]
```

PASO 6: Aplicar factor de relajación (lmbda)

```
    x[i] = lmbda * suma + (1.0 - lmbda) * old
```

Código en Python

```
# PASO 7: Validar precisión y estabilizar
    if centinela == 1 and x[i] != 0:
        ea = abs((x[i] - old) / x[i]) * 100
        if ea > es:
            centinela = 0
    iter = iter + 1
    if centinela == 1 or iter >= imax:
        break
# Retorna el resultado de la función
    return x, iter
```

Caso de prueba #1

Resolveremos un sistema de 3x3 con una matriz diagonalmente dominante y se usa un factor = 1.0

```
# --- DATOS DEL CASO DE PRUEBA 1 ---  
# Sistema:  
# 3x1      - 0.1x2 - 0.2x3 = 7.85  
# 0.1x1 + 7x2      - 0.3x3 = -19.3  
# 0.3x1 - 0.2x2 + 10x3 = 71.4
```

Caso de prueba #1

Asignamos los datos de la matriz a las variables

```
A = [[3.0, -0.1, -0.2], [0.1, 7.0, -0.3], [0.3, -0.2, 10.0]]  
B = [7.85, -19.3, 71.4]  
X_inicial = [0.0, 0.0, 0.0]
```

Cargamos las variables dentro de la funcio y imprimimos los resultados

```
resultado, iteraciones = gseid(A, B, 3, X_inicial, 100, 0.001, 1.0)  
  
print(f"--- Caso 1: Convergencia ---")  
print(f"Solución: {resultado}")  
print(f"Iteraciones: {iteraciones}")
```

Caso de prueba #1

Salida: Muestra los valores finales de x y el número de iteraciones necesarias para alcanzar la precisión deseada en condiciones ideales.

```
--- Caso 1: Convergencia ---  
Solución: [2.99999999980555687, -2.5000000000456044,  
7.0000000000049213]  
Iteraciones: 5
```

Caso de prueba #2

Resolveremos un sistema de 2×2 aplicando sobrerrelajación con $\lambda = 1.2$ para acelerar el cálculo y reducir los pasos repetitivos.

```
# --- DATOS DEL CASO DE PRUEBA 2 ---  
# Sistema:  
#  $10x_1 + 2x_2 = 12$   
#  $2x_1 + 10x_2 = 12$ 
```

Caso de prueba #2

Asignamos los datos de la matriz a las variables

```
A2 = [[10.0, 2.0], [2.0, 10.0]]
```

```
B2 = [12.0, 12.0]
```

```
X_inicial2 = [0.0, 0.0]
```

Cargamos las variables dentro de la funcio y imprimimos los resultados

```
# Usamos lambda = 1.2 para acelerar
```

```
resultado2, iteraciones2 = gseid(A2, B2, 2, X_inicial2, 50, 0.0001, 1.2)
```

```
print(f"\n--- Caso 2: Sobrerrelajación (Lambda 1.2) ---")
```

```
print(f"Solución: {resultado2}")
```

```
print(f"Iteraciones: {iteraciones2}")
```

Caso de prueba #2

Salida: Muestra la solución del sistema y evidencia cómo un factor de relajación mayor a 1.0 optimiza el tiempo de respuesta del algoritmo.

--- Caso 2: Sobrerrelajación (Lambda 1.2) ---

Solución: [0.99999999632087728, 0.99999999624383012]

Iteraciones: 10

Animación

1. Sistema de Ecuaciones Lineales

Conclusiones

El método de Gauss-Seidel es una herramienta eficiente para resolver sistemas de ecuaciones lineales mediante un proceso iterativo que mejora progresivamente la solución. Su capacidad de utilizar valores actualizados en cada paso le permite converger con mayor rapidez en muchos casos.

Sin embargo, su correcto funcionamiento depende de las características del sistema, como la diagonal dominante.

REFERENCIAS

Chapra, S. C., & Canale, R. P. (2015). Métodos numéricos para ingenieros (7.^a ed.) [PDF]. McGraw-Hill. Comprender paso a paso el método de Gauss-Seidel mediante una explicación visual y estructurada hasta su implementación computacional que facilite el aprendizaje del proceso iterativo y su implementación práctica.

Los métodos numéricos permiten soluciones aproximadas de problemas matemáticos complejos (Chapra & Canale, 2005).